

swmmx: A Python toolkit for building, editing, running, importing, visualizing, and exporting EPA SWMM models

Mohammadali Geranmehr ¹

¹ University of Sheffield, United Kingdom

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

swmmx is an open-source Python package for working with United States Environmental Protection Agency Storm Water Management Model (EPA SWMM) projects from Python. It provides a high-level interface for opening, creating, editing, validating, running, visualizing, importing, and exporting SWMM models. The package is designed for researchers, engineers, and data scientists who use SWMM in reproducible modelling workflows, scenario analysis, teaching, and applied research.

The package centres on a single model object:

```
from swmmx import swmm

m = swmm("examples/example.inp")
m.plot_layout()
lengths = m.get.conduit.length()
m.set.conduit.roughness(0.013)
m.run()
m.plot_timeseries.link.flow()
```

This object exposes a discoverable API for model access and modification, including `m.get`, `m.set`, `m.count`, `m.add`, `m.remove`, `m.import_csv`, `m.import_gis`, `m.export`, `m.validate`, `m.run`, and `m.runs`. The package supports both conventional batch simulation with `m.run()` and step-by-step simulation with `m.runs()`, which exposes the native SWMM step loop as a Python iterator. It also includes plotting tools for model layouts, result time series, and longitudinal profiles, as well as export tools for CSV, GIS, and Excel outputs.

Statement of need

EPA SWMM is widely used for modelling stormwater, wastewater, combined sewer, and urban drainage systems ([Rossman, 2015](#)). It is an established hydrologic and hydraulic simulation engine, but many research and engineering workflows require repeated model editing, scenario generation, automated simulation, data exchange with GIS or tabular datasets, and reproducible post-processing. These workflows are often difficult to manage through a graphical interface alone, especially when models must be generated or modified programmatically.

Several Python packages already support SWMM-related workflows. PySWMM provides Pythonic access to the SWMM engine and is particularly useful for simulation control and interaction during a run ([McDonnell et al., 2020](#)). `swmm_api` provides tools for reading, modifying, writing, and analysing SWMM input, report, and output files ([Pichler, 2024](#)). `swmmio` supports pandas-oriented access to SWMM input and result data for pre- and post-processing ([PySWMM](#)

32 [contributors, 2024a](#)). `swmm-toolkit` provides lower-level wrappers around SWMM solver and
33 output libraries ([PySWMM contributors, 2024b](#)).

34 `swmmx` addresses a complementary need: an integrated, high-level Python modelling interface
35 that covers the full SWMM model lifecycle in one consistent API. The goal is not only to run
36 SWMM from Python, but also to make model construction, inspection, editing, validation,
37 plotting, import, export, and iterative analysis accessible through a single object-oriented
38 workflow. This is useful for research applications such as scenario analysis, sensitivity and
39 uncertainty analysis, design and operational optimisation, model calibration workflows, teaching
40 examples, reproducible urban drainage studies, and data-driven integration with GIS and
41 monitoring datasets.

42 Software design and functionality

43 `swmmx` is designed around a single model object returned by the `swmm` constructor. The
44 constructor can open an existing SWMM input file or create a new model using SI or US unit
45 conventions:

```
m = swmm("examples/example.inp")
m = swmm()
m = swmm(new="SI", flow_unit="CMS")
m = swmm(new="US", flow_unit="GPM")
```

46 The public interface is organized into namespaces that mirror common modelling tasks.
47 Parameter access is provided through `m.get.<category>.<parameter>()`, while editing is
48 provided through `m.set.<category>.<parameter>()`. For example:

```
lengths = m.get.conduit.length()
one_length = m.get.conduit.length("P001")
flow = m.get.link.flow(ids=["P001", "P005"], format="df")
```

```
m.set.conduit.roughness(0.013)
m.set.conduit.roughness([0.013, 0.014], ids=["P001", "P005"])
```

49 The getter interface supports model input fields, derived values, attached records, and result
50 variables when simulation output is available. The setter interface exposes editable parameters
51 and raises clear errors when users attempt to modify read-only, derived, or result fields.

52 Model construction is supported through `m.add.<category>.<element_type>()`. For example,
53 a simple model can be assembled from scratch by adding nodes, links, a rainfall time series, a
54 rain gage, and a subcatchment:

```
m = swmm(new="SI")
```

```
m.add.node.junction("J1", x=0.0, y=0.0, invert_elevation=10, max_depth=3)
m.add.node.outfall("OUT1", x=100.0, y=0.0, invert_elevation=9, type="FREE")
```

```
m.add.link.conduit(
    "C1",
    from_node="J1",
    to_node="OUT1",
    length=100,
    roughness=0.013,
    shape="CIRCULAR",
    diameter=1.0,
)
```

```
m.add.time.time_series(
```

```

        "Rain1",
        data=[
            ("2026-01-01 00:00", 0.0),
            ("2026-01-01 00:05", 5.0),
        ],
    )

    m.add.hydrology.rain_gage(
        "RG1",
        format="INTENSITY",
        interval="00:05",
        source_type="TIMESERIES",
        time_series="Rain1",
    )

    m.add.hydrology.subcatchment(
        "S1",
        rain_gage="RG1",
        outlet="J1",
        x=0.0,
        y=0.0,
        area=1.0,
    )

    m.save("new_model.inp")

```

55 The add and remove interfaces validate IDs, required values, numeric values, enumerated
 56 options, references, and removal dependencies. Removing elements uses `m.remove.<category>.<element>`.
 57 unsafe removals are refused by default, while conservative cascades may be performed when
 58 `force=True` is used. Model edits set the in-memory model state as modified and mark previous
 59 simulation results as stale.

60 Simulation workflows

61 swmmx supports normal and step-wise simulation workflows. The `m.run()` method executes
 62 the SWMM model and makes result variables available through the result-aware getter and
 63 plotting interfaces. The `m.runs()` method exposes the simulation step loop as an iterator:

```

for step in m.runs():
    print(step.index, step.time, step.elapsed_days)

print(m.time.count_run())

```

64 This design supports progress monitoring, interactive workflows, dynamic inspection, and future
 65 extensions for control-oriented modelling. Pre-run time helpers, such as `m.time.vector()`
 66 and `m.time.count()`, reflect the expected report period. Post-run helpers, such as
 67 `m.time.vector_run()` and `m.time.count_run()`, use the actual period count read from the
 68 SWMM output file after the engine finishes.

69 swmmx uses lazy native-engine loading and supports bundled Windows and Linux engines, with
 70 a custom engine path option for other deployments. This allows users to work with SWMM
 71 models from Python while retaining access to the native SWMM simulation engine.

72 Import, export, and data exchange

73 Urban drainage models commonly interact with external asset databases, GIS layers, survey
74 tables, calibration datasets, and reporting tools. swmmx includes import and export interfaces
75 to support these workflows.

76 CSV import is exposed through:

```
m.import_csv.node.junction("junctions.csv")
m.import_csv.node.outfall("outfalls.csv")
m.import_csv.link.conduit("conduits.csv")
m.import_csv.hydrology.subcatchment("subcatchments.csv")
```

77 GIS import is exposed through:

```
m.import_gis.node.junction("junctions.shp")
m.import_gis.link.conduit("pipes.geojson")
m.import_gis.hydrology.subcatchment("subcatchments.gpkg", layer="subcatchments")
```

78 The importers normalize column names, apply common aliases, validate rows, and return
79 an ImportResult object. Users may provide explicit field maps when their source data use
80 project-specific naming conventions:

```
result = m.import_csv.link.conduit(
    "pipes.csv",
    field_map={
        "id": "PipeID",
        "from_node": "FromNode",
        "to_node": "ToNode",
        "length": "Length",
        "roughness": "ManningN",
        "diameter": "Diameter",
    },
)
```

81 The import API supports dry runs, update and upsert modes, error handling options, unknown-
82 field handling, and group-level shortcuts that dispatch rows by type. GIS import uses optional
83 geopandas and shapely dependencies.

84 Export is available through:

```
m.export.gis()
m.export.csv(path="exports/csv", elements="all", time_step=-1)
m.export.excel(path="exports", file_name="model_export.xlsx")
```

85 CSV exports write one table per file, Excel exports write one workbook with one sheet per
86 selected table, and GIS exports write spatial layers for nodes, links, subcatchments, and rain
87 gages. Exported tables can include model parameters and, when results are available, result
88 snapshots. CSV files generated by m.export.csv() are designed to import back cleanly where
89 possible, which supports round-trip editing and data exchange.

90 Visualization

91 swmmx provides built-in visualization tools using Matplotlib (Hunter, 2007). The layout plotter
92 draws mapped subcatchments, links, nodes, rain gages, and LID-related objects where available:

```
m.plot_layout(
    title="Drainage Network",
    legend=True,
```

```
        grid=True,  
        axis=True,  
    )
```

93 Layer dictionaries support static and data-driven symbology. Nodes, links, and subcatchments
94 can be styled by input parameters, result variables, or user-supplied mappings. For example,
95 links can be coloured by conduit roughness or plotted with result-driven widths. Legends and
96 sampled continuous swatches are created to make encoded values interpretable on the final
97 figure.

98 Time-series plotting is available through `m.plot_timeseries.<category>.<parameter>()`.
99 These functions read through the result API and can plot one or many object time
100 series. Longitudinal profile plotting is available through `m.plot_profile.nodes()`,
101 `m.plot_profile.links()`, and `m.plot_profile.longest()`, with hydraulic grade line and
102 water-depth overlays when simulation results are available. Plotting functions return Matplotlib
103 (`fig`, `ax`) objects, support `ax=` for composition, and support saving through `save_path` and
104 `save_format`.

105 Validation and reproducibility

106 The `m.validate()` interface checks common model issues, including duplicate IDs, missing
107 required options, invalid unit values, missing nodes or links, conduit endpoint problems, and
108 several cross-section and reference errors. The returned validation object exposes a summary
109 status and tabular representation:

```
validation = m.validate()  
print(validation.ok)  
print(validation.to_frame())
```

110 Validation does not replace engineering review, but it helps identify common input problems
111 before simulation or publication of results. The package also includes logging and cloning
112 helpers, as well as preservation-aware parsing and writing of SWMM input files. When possible,
113 `swmmx` preserves comments, unknown sections, and section order during `.inp` file round trips.
114 This is important for real engineering models, where files often include manual comments,
115 legacy sections, and formatting conventions that users do not want to lose during automation.

116 The package documentation includes executable examples and reference notebooks for the
117 getter, setter, add, remove, plot, import, and export interfaces. These examples are intended
118 to support both learning and regression-style checking of expected behaviours.

119 Relationship to existing software

120 The Python SWMM ecosystem includes several mature and useful packages. PySWMM is well
121 suited to simulation control and real-time interaction with the SWMM engine (McDonnell
122 et al., 2020). `swmm_api` provides extensive tools for SWMM input, report, and output file
123 automation (Pichler, 2024). `swmmio` gives users a pandas-oriented interface for model and
124 result processing (PySWMM contributors, 2024a). `swmm-toolkit` exposes lower-level solver
125 and output bindings (PySWMM contributors, 2024b).

126 `swmmx` is intended to complement these tools by providing a high-level, integrated modelling
127 environment. Its main contribution is a consistent API that covers model creation, editing,
128 validation, import, export, plotting, and simulation in one workflow. This makes it suitable for
129 users who need to build reproducible scripts around complete SWMM modelling tasks rather
130 than only one part of the workflow.

Availability

The source code for `swmmx` is available from GitHub at:

https://github.com/mgeranmehr/swmmx_dev

The package is available from the Python Package Index at:

<https://pypi.org/project/swmmx/>

Software status and responsible use

`swmmx` is under active development and testing. Users are encouraged to use the latest available release and to report unexpected behaviour through the project issue tracker.

The package is a programmatic toolkit for EPA SWMM and assumes that users have appropriate knowledge of hydrology, hydraulics, SWMM input conventions, and the EPA SWMM simulation engine. Simulation results should be checked carefully before use in research, design, planning, or operational decision-making. As with any modelling software, professional engineering judgement and independent verification are required for critical applications.

Acknowledgements

The author acknowledges the developers and maintainers of EPA SWMM and the wider open-source Python scientific computing ecosystem. `swmmx` builds on widely used Python tools including NumPy (Harris et al., 2020), pandas (McKinney, 2010; team, 2020), Matplotlib (Hunter, 2007), and NetworkX (Hagberg et al., 2008). The author also acknowledges the developers of existing Python SWMM packages, including PySWMM, `swmm_api`, `swmmio`, and `swmm-toolkit`, which have helped establish and grow the Python ecosystem for SWMM modelling.

AI usage disclosure

AI tools were used during software development to assist with repetitive programming, documentation, and debugging tasks. For the manuscript, AI tools were used for grammar checking and language refinement. All outputs were reviewed and validated by the author.

References

- Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. *Proceedings of the 7th Python in Science Conference*, 11–15.
- Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- McDonnell, B., Ratliff, K., Tryby, M., Wu, J., & Mullanpudi, A. (2020). PySWMM: The Python Interface to Stormwater Management Model (SWMM). *Journal of Open Source Software*, 5(52), 2292. <https://doi.org/10.21105/joss.02292>
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.

- 169 Pichler, M. (2024). *swmm_api*: Python package for EPA SWMM input, report, and output
170 files. https://github.com/MarkusPic/swmm_api
- 171 PySWMM contributors. (2024a). *swmmio*: Tools for interacting with EPA SWMM models
172 and results. <https://github.com/pyswmm/swmmio>
- 173 PySWMM contributors. (2024b). *swmm-toolkit*: Python wrappers for SWMM solver and
174 output libraries. <https://github.com/pyswmm/swmm-python/tree/master/swmm-toolkit>
- 175 Rossman, L. A. (2015). *Storm Water Management Model User's Manual Version 5.1*. United
176 States Environmental Protection Agency. [https://www.epa.gov/water-research/storm-](https://www.epa.gov/water-research/storm-water-management-model-swmm)
177 [water-management-model-swmm](https://www.epa.gov/water-research/storm-water-management-model-swmm)
- 178 team, T. pandas development. (2020). *pandas-dev/pandas*: Pandas (latest). Zenodo.
179 <https://doi.org/10.5281/zenodo.3509134>

DRAFT